

# Design Justification Report: A 128-MAC INT8 Convolution Accelerator

ECE 410/510 HW4AI – Milestone 4

Kay Hynes

June 7, 2026

## 1. Problem and motivation

This project accelerates the **3x3 INT8 convolution** kernel that dominates YOLO-nano object-detection inference. The motivation is grounded in the M1 profiling data, not intuition. Profiling a representative layer (input 52x52x64, kernel 3x3x64x128, output 52x52x128) on an Apple M1 Pro with a vectorized NumPy im2col+GEMM path showed that the **3x3 convolution accounts for 95.6% of total layer execution time** (2.275 s of 2.379 s across 15 runs; `project/heilmeier.md`). The kernel runs in a median of **160.9 ms/layer** (mean 158.4 ms, std 7.4 ms; `project/m1/sw_baseline.md`), delivering only **2.48 GFLOP/s** – about **1.2% of the M1 Pro’s ~200 GFLOP/s FP32 NEON peak**. The layer performs 398,721,024 FLOPs (199,360,512 INT8 MACs).

The limit of current practice for this workload is the mismatch between a general-purpose core and a kernel that is a tight, regular multiply-accumulate reduction. The CPU spends energy on instruction fetch, out-of-order control, cache coherence, and an INT8->INT32 cast pipeline that the kernel does not need. A fixed-function INT8 MAC array does only the useful arithmetic, so it can do it faster per joule and is a natural fit for an edge camera or sensor that runs the convolution layers of YOLO-nano without a GPU. Custom hardware is justified specifically because (a) one kernel dominates runtime, so Amdahl’s law gives a large addressable speedup, and (b) the kernel is overwhelmingly compute-bound (Section 2), so adding parallel multipliers translates almost directly into throughput.

## 2. Roofline analysis

The arithmetic intensity (AI) of the kernel is **672.5 FLOP/byte**: 398,721,024 FLOPs over 592,896 operand bytes (input feature map 173,056 B + weights 73,728 B + output 346,112 B, counted once with no interface reuse; `project/m1/interface_selection.md`). Figure 1 plots this on log-log roofline axes (FLOP/byte vs. GFLOP/s), identical to the M1 roofline axes.

This AI is far to the right of every relevant ridge point. The M1 Pro CPU ridge (peak compute / DRAM bandwidth) is ~1.0 FLOP/byte; the accelerator ridge, using its ~32 GB/s on-chip line-buffer/SRAM bandwidth against a 64 GFLOP/s target ceiling, is ~2.0 FLOP/byte. At AI = 672, the kernel sits **two-to-three orders of magnitude past both ridges**, so it is **compute-bound**

on the CPU and would remain compute-bound on the accelerator. The bottleneck only shifts to memory if AI were to drop below  $\sim 2$  FLOP/byte, which would require destroying on-chip reuse.

This analysis directly shaped the architecture. Because performance is bounded by available multipliers rather than by bandwidth, the design **spends its silicon budget on parallel MAC lanes** (128 of them) rather than on large caches or a wide off-chip interface. It also justified INT8: shrinking operands does not help a compute-bound kernel via bandwidth, but it shrinks each multiplier  $\sim 4\times$  versus FP32, letting more MACs fit in a given area – the lever that actually moves a compute-bound design. Figure 1 confirms the outcome: the measured M4 point sits on the flat (compute) portion of the accelerator roofline at the kernel’s AI, vertically above the software baseline by the measured speedup, and nowhere near the bandwidth-limited diagonal.

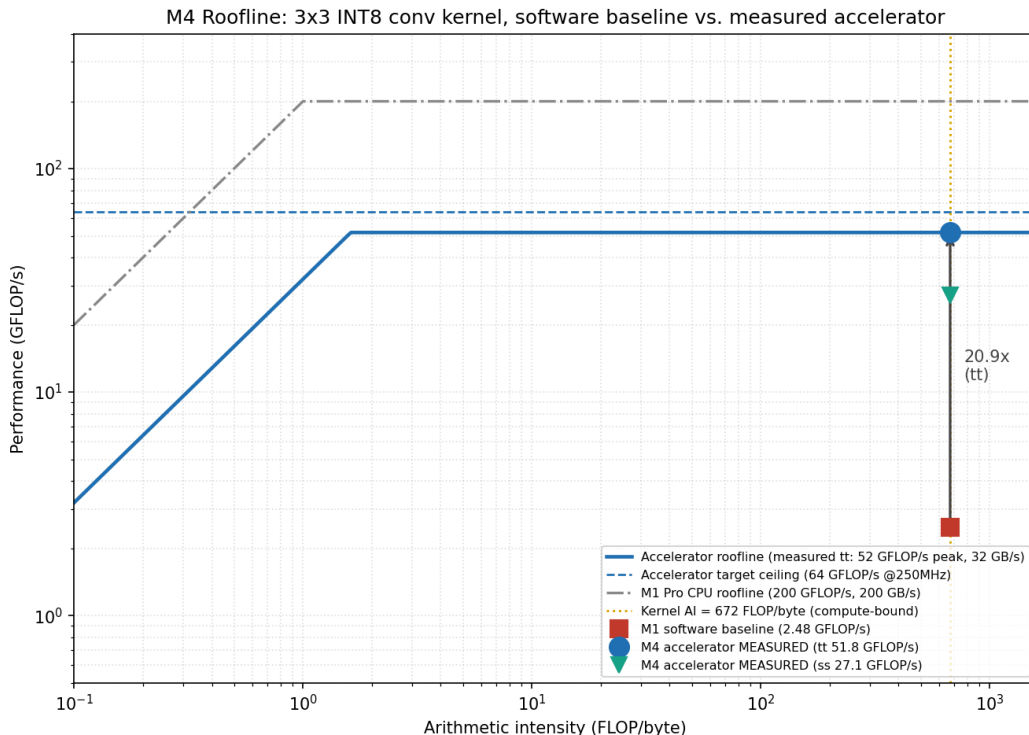


Figure 1: Roofline (log-log): the 3x3 INT8 conv kernel at AI = 672 FLOP/byte sits on the compute-bound ceiling. The measured M4 accelerator points (tt 51.8 GFLOP/s, ss 27.2 GFLOP/s) are plotted against the M1 software baseline (2.48 GFLOP/s); the arrow marks the 20.9x typical-corner speedup.

### 3. Precision and data format

The datapath uses **signed INT8 activations and signed INT8 weights with signed INT32 accumulation**, matching the M1 baseline and the M2 precision study (project/m2/precision.md). Each operand is a two’s-complement integer in  $[-128, 127]$ ; each product is a signed 16-bit value; products are sign-extended to 32 bits and summed in a 32-bit accumulator. There is no internal fixed-point fractional format – the RTL is a pure integer dot-product engine, and any real  $\rightarrow$  INT8 scaling is a host-side concern (symmetric scale  $1/127$ ).

INT8 is the right precision for an edge object-detection convolution. M2’s error analysis quantified acceptability: over 1,000 deterministic 9-tap dot-product samples (seed 510), the INT8 path versus an FP32 reference gave **mean absolute error 0.00451, RMS error 0.00557, max absolute error 0.0178**, with the mean error equal to **0.54% of the mean absolute reference magnitude** (inputs scaled to  $\sim[-1, 1]$ ). For a datapath-validation milestone this is acceptable; a shipping accelerator should still be checked against a labeled YOLO-nano calibration set before any task-accuracy claim, which M2 explicitly flagged. FP16 was rejected as spending more area/bandwidth than an edge detector needs; INT4 was rejected as too risky without a quantization-aware-trained model or calibration set in the repository. The 32-bit accumulator width is chosen so the worst-case 576-element reduction of INT8 products ( $|\text{product}| \leq 16,384$ ,  $\text{sum} \leq \sim 9.4\text{M}$ ) cannot overflow.

## 4. Dataflow and architecture

**Dataflow: output-stationary, weight-streaming.** Each of the 128 lanes owns one output channel and keeps that channel’s partial sum **resident (stationary) in its accumulator** across the entire 576-element reduction; weights stream in one per lane per cycle, and a single INT8 activation is **broadcast** to all lanes each cycle. Figure 3 shows this dataflow. Output-stationary fits the kernel because a  $3 \times 3 \times 64$  convolution reduces 576 products into one output value per channel: keeping the accumulator in place avoids writing and re-reading partial sums, and broadcasting the shared activation exploits the fact that all 128 output channels consume the same input patch element. This is the form the RTL actually implements (`rtl/mac_array.sv` header and body), not an aspiration.

**Compute engine.** `mac_array` is a parameterized array (`NUM_MAC=128`, `DATA_WIDTH=8`, `ACC_WIDTH=32`). Each lane is a **3-stage pipeline**: Stage A captures the activation and the lane’s weight; Stage B performs the signed  $8 \times 8 \rightarrow 16$ -bit multiply into a registered product; Stage C sign-extends to 32 bits and accumulates, emitting the result when the **last** tag arrives. A shared control pipeline carries `valid/first/last` tags alongside the data so that accumulations for back-to-back output pixels stream with **no bubble**. **first** clears the accumulator (starts a new pixel); **last** emits the channel result. This explicitly removes the M3 single-lane critical path (a runtime tap-select mux  $\rightarrow$  multiply  $\rightarrow$  add in one cycle) and its restart penalty.

**Memory hierarchy and data path.** In the intended FPGA-SoC deployment, weights live in on-chip SRAM (the 72 KB weight set) feeding the per-lane weight ports, activations arrive through a line-buffer/stream, and results drain to an output buffer; this on-chip path supplies  $\sim 32$  GB/s, far above demand (Section 5). The integrated top (Figure 2) wires the AXI4-Stream interface `stream_if` to the compute core `compute_core` (a transparent wrapper of `mac_array`). The honest scope note: the **standalone OpenLane run synthesized the compute core (`mac_array`) directly** (`synth/config.json`); `stream_if`, `compute_core`, and `top` are the integration RTL, verified end-to-end in simulation (Section 6) but not separately re-synthesized, because the compute fabric dominates area, timing, and power and because exposing every weight/result as a chip pin created a floorplan artifact (Section 9).

## 5. Hardware interface

The interface is **AXI4-Stream**, selected in M1 as the native streaming transport on an FPGA-SoC host (e.g., Zynq UltraScale+), where an ARM core orchestrates the model and the programmable logic holds the accelerator. M4 realizes it as `rtl/interface.sv` (`stream_if`): an input

Figure 2. M4 accelerator block diagram: host -> AXI4-Stream interface -> 128-lane MAC compute core

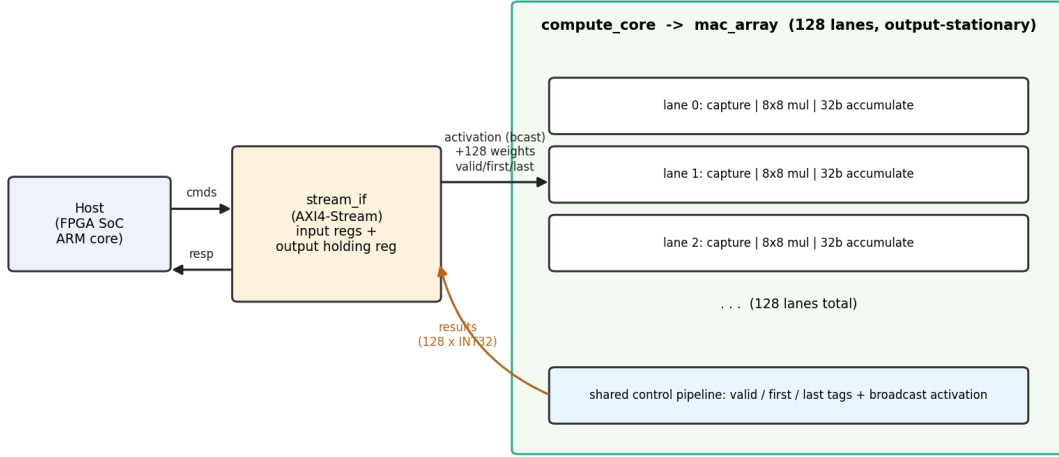


Figure 2: Block diagram of the integrated accelerator top: an FPGA-SoC host drives the AXI4-Stream interface `stream_if`, which feeds the broadcast activation, 128 per-lane weights, and valid/first/last tags into the 128-lane `compute_core/mac_array`; 128 INT32 channel results drain back through the interface.

Figure 3. Output-stationary dataflow: activations broadcast, weights stream, accumulators stay resident

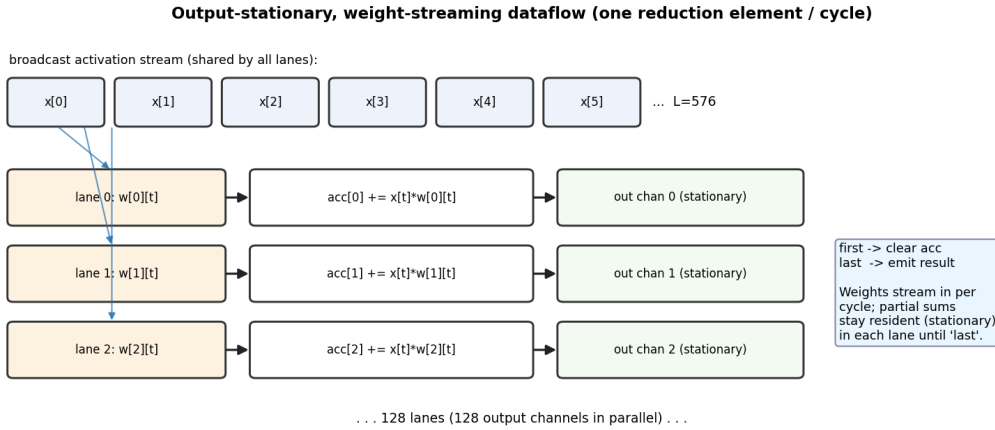


Figure 3: Output-stationary, weight-streaming dataflow: one INT8 activation is broadcast per cycle, each lane streams its own weight, and partial sums stay resident in per-lane accumulators until the `last` tag emits the channel result.

stream carries one reduction-element beat per cycle (broadcast activation + 128 packed weights + **first/last**), registered once before the array; an output stream drains one 128-channel INT32 result per completed pixel, held until the host asserts **m\_tready**. This is the scale-up of the M2/M3 single-word command interface (**axis\_interface**) to a wide streaming-data port that can feed all 128 lanes.

**Effective bandwidth at target throughput.** At the typical (tt) measured operating point (7.70 ms/layer, Section 8), the layer moves 592,896 operand bytes, so the *required* off-accelerator bandwidth is only **~0.077 GB/s**; even at the fast corner it is **~0.13 GB/s**. A 32-bit @ 100 MHz AXI4-Stream link is rated at **0.4 GB/s** – a **4.3x headroom** at the M1 target throughput (**interface\_selection.md**) – and the on-chip operand path provides **~32 GB/s**.

**Is the design interface-bound? No, and it is quantified.** Required bandwidth (**~0.08-0.13 GB/s**) is 3-400x below the available interface/on-chip bandwidth, and the kernel AI (672 FLOP/byte) is **~300x** above the accelerator ridge (**~2 FLOP/byte**). On the roofline (Figure 1) the operating point lies on the flat compute ceiling, not the bandwidth diagonal. The accelerator is **compute-bound (frequency-bound), not interface-bound**.

## 6. Verification

Correctness was verified by **cycle-accurate simulation against an independent golden reference**, building on the M2 and M3 testbenches. M2 verified the single INT8 dot-product **compute\_core** and the AXI4-Stream **interface** separately (**tb\_compute\_core.sv**, **tb\_interface.sv**); M3 verified the integrated single-lane **top** end-to-end through host-side AXI4-Stream transactions only (**m3/tb/tb\_top.sv**, **PASS: m3 end-to-end cosim**). M4’s **tb/tb\_top.sv** continues this contract at 128-lane scale: it drives the integrated **top exclusively through the AXI4-Stream ports** (never poking the array) and streams a representative slice of the dominant convolution – 8 output pixels, each a full L=576 reduction, 128 channels – back-to-back with no bubbles.

**What the tests cover.** (a) *Functional correctness*: an independent SystemVerilog reference re-computes all  $8 \times 128 = 1,024$  channel results and compares every one; the run reports **errors=0** (**sim/final\_run.log**). This exercises the signed 8x8 multiply, sign-extension, 32-bit accumulation, the **first** clear and **last** emit semantics, and the interface’s input registering and output holding. (b) *Sustained throughput / no-bubble streaming*: the testbench measures cycles for the streamed region, confirming **127.861 MAC/cycle (99.89% of 128)**, which verifies that back-to-back pixel accumulations incur no restart penalty – the specific M3 defect this design targets. (c) *Handshake behavior*: the testbench holds **m\_tready** and honors **s\_tready** backpressure, and Figure 4 (extracted from the run’s VCD) annotates one end-to-end transaction: the input **s\_tvalid/s\_tready** handshake and **first** tag entering the array, and the first 128-channel result draining on **m\_tvalid/m\_tready** after the 576-element reduction. The M2 precision study (Section 3) provides the numerical-accuracy verification that complements this logical verification.

## 7. Synthesis results

Synthesis used **OpenLane 2 v2.3.10 on the sky130A / sky130\_fd\_sc\_hd PDK**, with the compute core (**mac\_array**, **NUM\_MAC=128**) as the design and a 4.0 ns (250 MHz) target clock (**synth/config.json**, **synth/openlane\_run.log**).

M4 128-MAC accelerator: one end-to-end transaction (top = stream\_if + compute\_core/mac\_array)

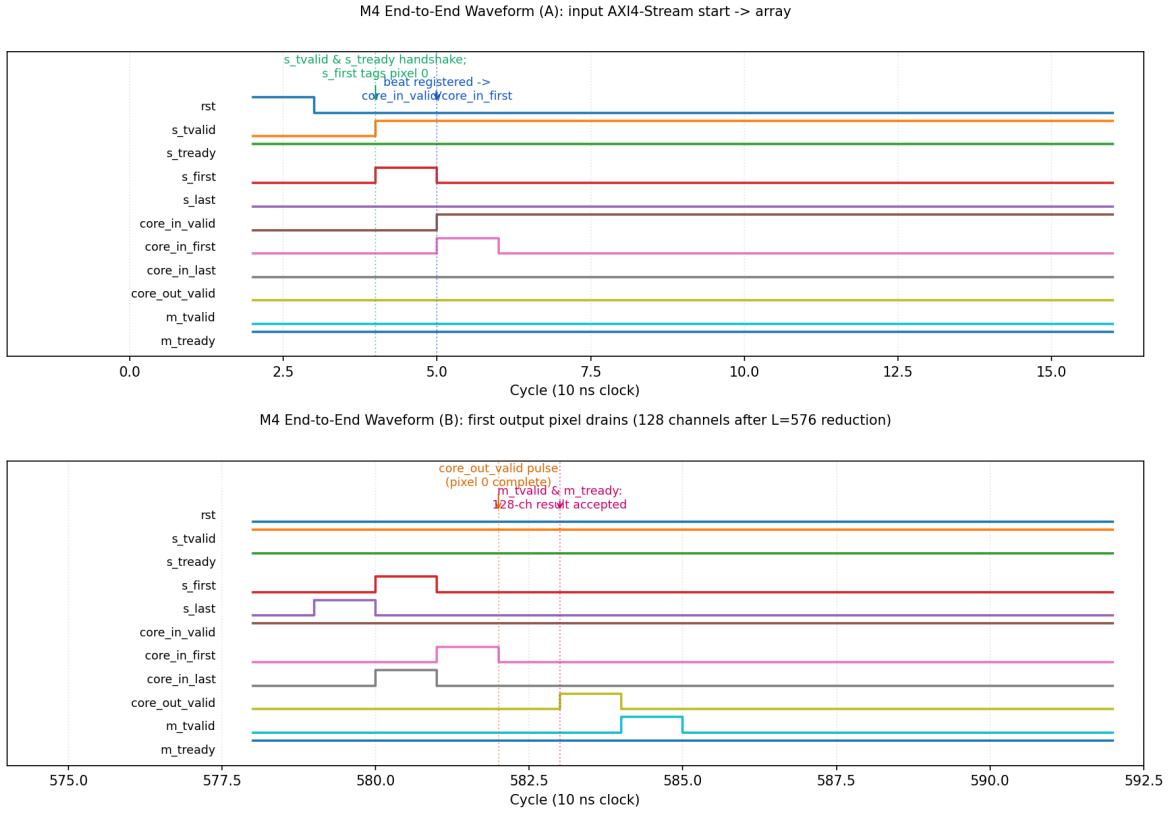


Figure 4: Annotated end-to-end waveform from the final simulation VCD. Panel A: the input AXI4-Stream handshake and **first** tag entering the array. Panel B: after the L=576 reduction, **core\_out\_valid** pulses and the 128-channel result is accepted on **m\_tvalid/m\_tready**.

**Area (synth/area\_report.txt).** The mapped design is **99,710 cells, 1,065,403  $\mu\text{m}^2$  (1.065  $\text{mm}^2$ )**, 0 unmapped cells, 0 inferred latches. A standalone yosys cross-check reports 11,279 flip-flops, matching 128 lanes x (8b weight + 16b product + 32b accumulator + 32b result) plus shared control tags. The **dominant area contributors are the 128 signed 8x8 multipliers and the 128 32-bit accumulators** (xnor2/nor2/nand2/xor2/maj3 arithmetic cells), with sequential elements  $\sim 29\%$  of area. Versus the M3 single lane (2,000 cells, 23,130  $\mu\text{m}^2$ ) this is  $\sim 46\times$  – sub-128x because the array drops the per-lane AXI command glue and the runtime tap-select mux and shares one control pipeline.

**Timing (synth/timing\_report.txt).** The pre-PNR full-array STA reports a catastrophic -387 ns setup WNS, but this is **not a real Fmax**: the worst path is the broadcast **b\_valid** net at **fanout 5,881** driving an unbuffered inverter (single-gate delay 333.7 ns) before any buffer insertion or clock-tree synthesis. The pipeline logic depth is short; the violation is pure interconnect fan-out. To get a meaningful number, a single placed pipelined lane (shared by all 128 lanes) was taken through placement + STA: it closes at **+1.05 ns /  $\sim 339$  MHz (ff), -0.94 ns /  $\sim 202$  MHz (tt), and -5.42 ns /  $\sim 106$  MHz (ss, sign-off)**. The design therefore **does not meet 250 MHz at the slow corner** – it reaches  $\sim 106$  MHz (ss) /  $\sim 202$  MHz (tt), still a  $1.75\times$  slow-corner improvement over the M3 unpipelined lane (60.6 MHz). The limiter is the **32-bit accumulate adder carry chain** (acc + prod\_ext), the single dominant logic stage.

**Power (synth/power\_report.txt).** OpenROAD pre-PNR estimates (default switching activity) are **193.5 mW (ss), 255.4 mW (tt), 342.5 mW (ff)**. At tt the split is 131.1 mW sequential (51.3%) and 124.3 mW combinational (48.7%) – the per-lane accumulators and pipeline registers dominate sequential power. This is higher than the crude 133 mW linear projection precisely because the real array carries full 32-bit accumulators per lane. It is a pre-PNR / default -activity figure (no clock tree), to be re-annotated from a workload VCD after routing.

## 8. Benchmark results

Using the measured 127.861 MAC/cycle and the post-synthesis frequencies, the full 2,704-pixel layer extrapolates as follows (bench/benchmark\_data.csv, bench/benchmark.md); the metric is GFLOP/s, identical to the M1 baseline:

| Config                  | Freq    | Layer time | Throughput   | Speedup       |
|-------------------------|---------|------------|--------------|---------------|
| M1 software baseline    | –       | 160.9 ms   | 2.48 GFLOP/s | 1.00x         |
| M4 accel, ss (sign-off) | 106 MHz | 14.69 ms   | 27.2 GFLOP/s | <b>10.96x</b> |
| M4 accel, tt (typical)  | 202 MHz | 7.70 ms    | 51.8 GFLOP/s | <b>20.89x</b> |
| M4 accel, ff (fast)     | 339 MHz | 4.60 ms    | 86.7 GFLOP/s | 34.98x        |

The **measured speedup is 10.96x (guaranteed sign-off) to 20.89x (typical)** over the M1 baseline. Energy per layer (corner power x runtime) is 2.84 mJ (ss) / 1.97 mJ (tt), i.e. **140-203 GFLOP/s/W**. Against an assumed 15 W M1 Pro CPU active power ( $\sim 2,414$  mJ/layer, 0.165 GFLOP/s/W) this is roughly **1,200x better energy efficiency** – reported as an order-of-magnitude estimate, gated on the 15 W assumption and the pre-PNR power figure, per the

checklist’s “optional but valued” energy item.

**Gap between measured and theoretical.** The M1 design target was 64 GFLOP/s at 250 MHz. Compute *utilization* is essentially ideal (99.89% of 128 MAC/cycle), so the gap is **entirely clock frequency**: the placed datapath closes at 106 MHz (ss) / 202 MHz (tt) rather than 250 MHz because the 32-bit ripple-carry accumulate adder is the critical path. Measured tt throughput (51.8 GFLOP/s) is 0.81x of target; ss (27.2) is 0.42x. Figure 1 plots the measured tt and ss points – the real measured values, not the M1 hypothetical 64 GFLOP/s point. The roofline confirms the design is frequency-bound, not bandwidth-bound, so the remedy is faster timing closure (Section 9), not more bandwidth.

## 9. What did not work

This design had real setbacks; the following are specific.

**(1) The full 128-lane array could not be placed-and-routed as a standalone block.** Synthesizing `mac_array` with every weight and result as a top-level pin exposed **5,134 chip pins**; floorplanning failed first with global-placement overflow at a tight die, then with PPL-0024 (IO pins exceed die-perimeter routing tracks). What I learned: a compute array is not a chip – in a real accelerator those operands come from on-chip SRAM and accumulator buffers, not pads. What I would do differently: wrap the array with on-chip weight SRAM and an output buffer so the synthesized block has a realistic pin count, then PnR the wrapped block. As a result, the reported area/power are from the full array’s pre-PNR synthesis, and Fmax is from a placed single lane (the per-lane logic the 128 lanes share), which is documented rather than hidden.

**(2) Pre-PNR timing was meaningless due to broadcast fan-out.** My first STA reported -387 ns WNS. I initially read this as a logic-depth failure; it was actually a single unbuffered broadcast net (`b_valid`) at fanout 5,881 with ~81 ns slew – an interconnect artifact that CTS/buffer insertion fixes. The lesson: pre-PNR STA on high-fanout broadcast control is not a Fmax, and a post-placement STA is required. What I would do differently: add a registered fan-out tree that re-broadcasts activation/valid/first/last into banks of ~16 lanes so no net drives more than ~17 sinks (`project/remaining_tasks.md` task 2).

**(3) The 250 MHz / 64 GFLOP/s target was optimistic.** The placed datapath closes at only ~106 MHz (ss). The cause is concrete: the worst path starts at the Stage-B product register and runs through the **32-bit accumulate adder carry chain**. What I would do differently, and the highest-value next step, is to replace the ripple-carry accumulator with a **carry-save accumulator** – keep the running sum in redundant (sum, carry) form across the reduction and do a single carry-propagate add only on `last` – removing the per-cycle 32-bit carry propagation from the inner loop (`remaining_tasks.md` task 1).

**(4) Power was higher than the linear projection (~2x).** The CF09 estimate scaled the single lane by 128 to ~133 mW; the real array is ~255 mW (tt) because full 32-bit accumulators and pipeline registers per lane add ~131 mW of sequential power that the linear model ignored. The lesson: per-lane state, not just multipliers, drives array power; and the current number is pre-PNR with default activity, so it must be re-estimated from a workload VCD after CTS (`remaining_tasks.md` task 3) before any final energy claim is made.

What did work, and is fully confirmed by measurement: the output-stationary, weight-streaming dataflow sustains 127.861 of 128 MAC/cycle with zero functional errors, eliminating the M3 restart



bubble. The open risk is timing closure, not parallelism or correctness.

---

*Figures: Figure 1 figures/fig1\_roofline.png; Figure 2 figures/fig2\_block\_diagram.png;  
Figure 3 figures/fig3\_dataflow.png; Figure 4 figures/fig4\_waveform.png.*